

Spec Driven Development (SDD) com Claude Code

Uma jornada em três aulas: por que a especificação existe, do que ela é feita, e o ciclo completo que a transforma em código verificado.

Módulo: SDD com Claude Code · Três aulas sequenciais – 01 leva a 02, que leva a 03

- 01 · Por que existe
- 02 · Anatomia
- 03 · O ciclo

Uma jornada em três aulas

Este documento resume, em formato executivo, o módulo Spec Driven Development (SDD) com Claude Code — três aulas em sequência, cada uma construída sobre a anterior. Diferente de um catálogo de recursos independentes, aqui a ordem importa: a Aula 01 estabelece o motivo, a 02 define do que a spec é feita, e a 03 fecha o ciclo mostrando quando e como cada peça entra em ação.

01

Por que existe o SDD

O custo de "codar no chute" e a virada de chave: a especificação passa a ser a fonte da verdade, o código vira artefato derivado dela.



02

Anatomia de uma Especificação

As seis peças que compõem uma spec que funciona, a fronteira entre "o quê" e "como", e a regra de ouro para saber se ela está pronta.



03

O ciclo e as fases do SDD

As seis fases — de princípios a verificação —, por que separar o pensar do fazer reduz retrabalho, e o tamanho certo de um ciclo.

Como ler: as três aulas formam uma progressão, não um catálogo — cada dossiê a seguir termina apontando para o próximo.

Do "codar no chute" à spec como fonte da verdade

Pedir código sem contexto ("faz um sistema de login pra mim") transfere decisões de produto para o agente — em silêncio, sem registro, e só se descobre o resultado depois de centenas de linhas escritas. O SDD inverte a hierarquia: a spec vira a fonte da verdade (o "o quê" e o "por quê") e o código passa a ser um artefato derivado dela (o "como").

O PROBLEMA

"Codar no chute" — mesmo pedido, resultados diferentes a cada vez

A VIRADA

Modelo antigo: código → (talvez) doc. Modelo SDD: spec → plano → código

DOC MORTA

Escrita depois, ninguém a lê, desatualiza em silêncio

SPEC EXECUTÁVEL

Escrita antes, é o input do agente — divergir quebra o próximo ciclo

```
> faz um sistema de login pra mim
// o agente agora precisa adivinhar: e-mail/senha? OAuth? sessão?
// 400 linhas depois: "não era bem isso"
```

custo de corrigir: na spec = segundos · em produção = uma sprint

- "Vai me deixar mais lento" — mais lento no primeiro passo, bem mais rápido no total: troca-se 5 min de escrita por 40 de retrabalho.
- "É burocracia disfarçada" — burocracia é o que ninguém consome; a spec é consumida pelo agente, pelo plano e pela verificação.
- "O modelo é esperto, ele entende" — e por isso preenche a lacuna com algo plausível, não necessariamente com o que você queria.

Seis peças, uma fronteira e a regra de ouro

A spec cuida do comportamento observável, das regras de negócio e dos critérios de sucesso — o "o quê / por quê". Nomes de classe, estrutura de pastas e escolha de algoritmo são "como" e ficam com o agente, salvo quando são restrição técnica justificada (ex.: "tem que ser PostgreSQL porque o time já opera Postgres").

1. OBJETIVO

O "por quê" em duas linhas: qual problema resolve, para quem

2. REQUISITOS FUNCIONAIS

Comportamentos observáveis — um requisito por linha

3. REQUISITOS NÃO-FUNCIONAIS

Desempenho, segurança, persistência — o que mais some das specs

4. CRITÉRIOS DE ACEITE

Peça central: afirmação testável com sim ou não, sem opinião

5. ESCOPO / FORA DE ESCOPO

Delimita o incremento; contém o crescimento não pedido

6. RESTRIÇÕES TÉCNICAS

"Como" já decidido e obrigatório — sempre com o motivo

```
✓ POST /transacoes com valor 0 retorna 422
✓ dados continuam lá depois de reiniciar o servidor
x a API deve ser rápida           // rápida quanto?
x tratar os erros direito         // quais erros? qual resposta?
```

Regra de ouro: se uma pessoa nova no time conseguisse implementar lendo só a spec — sem perguntar nada — então o agente também consegue.

- Adjetivo no lugar de número ("rápido", "seguro") — todo adjetivo de qualidade vira número ou vira nada.
- Só o caminho feliz — falta a linha "e se der errado?" para cada requisito.
- Spec grande demais — uma spec por incremento, não por produto inteiro.
- Spec que é código disfarçado — se não é observável de fora e não é restrição justificada, sai da spec.

Seis fases, um loop iterativo

Princípios → especificar → planejar → tarefas → implementar → verificar. Quatro fases de pensamento para uma de código — porque escrever código nunca foi a parte difícil. Quando pensamento e execução se misturam, cada mudança de ideia descarta código já escrito; separados, o que se descarta é texto.

FASE 0 · PRINCÍPIOS

CLAUDE.md — regras permanentes do projeto

FASE 1 · ESPECIFICAR

spec.md — o "o quê", decide sem decidir o "como"

FASE 2 · PLANEJAR

plano técnico — arquivos, ordem, riscos

FASE 3 · TAREFAS

lista ordenada, verificável passo a passo

FASE 4 · IMPLEMENTAR

código + commits — única fase que gera código

FASE 5 · VERIFICAR

critérios de aceite checados contra a spec

```
// especificar → planejar → (você ajusta o rumo aqui) → tarefas
> plano: 1 modelo, 1 migração, 2 rotas
x você revisa: "categoria inexistente devolve [], não 404 — está na spec"
// implementar → verificar contra a spec
✓ GET /transacoes?categoria=alimentacao → só alimentação
```

- O fluxo é iterativo, não linear — voltar à spec quando o plano revela uma decisão não tomada é o processo funcionando, não uma falha.
- Sinal de alerta: discutir "o quê" durante a implementação é a spec pedindo para ser atualizada.
- Tamanho certo de ciclo: cabe em 4 a 6 critérios de aceite e é demonstrável ao final — nem "adicionar um campo", nem "fazer o app inteiro".

O que cada fase produz

FASE	O QUE ACONTECE	ARTEFATO QUE PRODUZ
0 · Princípios	Regras permanentes do projeto: stack, padrões, convenções — vale para todos os ciclos.	CLAUDE.md
1 · Especificar	Descreve o incremento: objetivo, requisitos, escopo, critérios de aceite. Decide o O QUÊ.	spec.md
2 · Planejar	Traduz a spec em abordagem técnica: arquivos, ordem, decisões, riscos. O COMO aparece aqui.	plano técnico
3 · Tarefas	Quebra o plano em passos pequenos, ordenados e verificáveis isoladamente.	lista de tarefas
4 · Implementar	Executa as tarefas e escreve o código — a única fase que produz código.	código + commits
5 · Verificar	Confere o resultado contra a spec: critérios de aceite, testes, revisão.	critérios checados

A JORNADA COMPLETA, EM UMA FRASE POR AULA

- **01 · Por que existe** — sem spec, decisões de produto são tomadas em silêncio pelo agente; com spec, a decisão é sua e fica registrada.
- **02 · Anatomia** — a spec separa o "o quê/por quê" do "como" e se apoia em critérios de aceite verificáveis, não em adjetivos.
- **03 · O ciclo** — seis fases ligam a spec ao código: pensa-se em texto antes de escrever, e volta-se à spec sempre que o rumo desvia.

AULAS COMPLETAS DESTE MÓDULO

Spec Driven Development (SDD) com Claude Code/Por que existe o SDD/aula-f1-por-que-sdd-existe.html

Spec Driven Development (SDD) com Claude Code/Anatomia de uma Especificacao/aula-f2-anatomia-de-uma-especificacao.html

Spec Driven Development (SDD) com Claude Code/O ciclo e as fases do SDD/aula-f3-o-ciclo-e-as-fases.html

Resumo consolidado a partir do conteúdo das três aulas em HTML do módulo "Spec Driven Development (SDD) com Claude Code", parte da Formação Claude Code 2026.